

beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable **beamer** overlay specifications. In particular, they allow to replace raw numbers in **beamer** `<...>` overlay specifications by logical identifiers. Demonstration files are [available for download](#) as part of the [development repository](#). As a side effect, this is another solution to this [latex.org forum query](#).

Contents

1 Implementation

Identify the internal prefix (L^AT_EX3 DocStrip convention).

```
1 <@@=bnvs>
```

1.1 Package declarations

```
2 \NeedsTeXFormat{LaTeX2e}[2025/01/01]
3 \ProvidesExplPackage
4   {beanoves}
5   {2025/06/03}
6   {1.0}
7   {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string **beanoves** or containing the case insensitive string **bnvs** delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into **beamer** specifications based on this data model.

The development is made easier due to a facility layer over `expl`.

```
8 \regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
9   \A\s* (?:
```

- $2 \rightarrow V$

```

10      ( [^:]+? )

      • 3, 4, 5 → A : Z? or A :: L?

11      | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )

      • 6, 7 → ::(L:Z)?

12      | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )

      • 8, 9 → :(Z::L)?

13      | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
14    )
15    \s* \Z
16  }

```

1.3 Main scanner

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into L^AT_EX.

1.3.1 Grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters al have category other. We start with basic entities.

1 $\langle \text{space} \rangle$	::= Horizontal white space characters
2 $\langle \text{lower} \rangle$	::= a lowercase letter
3 $\langle \text{upper} \rangle$	::= an uppercase letter
4 $\langle \text{digit} \rangle$	::= a decimal digit
5 $\langle \text{sign} \rangle$	::= '-' '+'
6 $\langle \text{comma} \rangle$	::= $\langle \text{space} \rangle$ ',' $\langle \text{space} \rangle$
7 $\langle \text{equal} \rangle$	::= $\langle \text{space} \rangle$ ('=' '?=' '"=' '+=') $\langle \text{space} \rangle$
8 $\langle \text{alpha} \rangle$	::= $\langle \text{lower} \rangle$ $\langle \text{upper} \rangle$ '_'
9 $\langle \text{alnum} \rangle$	::= $\langle \text{alpha} \rangle$ $\langle \text{digit} \rangle$
10 $\langle \text{index} \rangle$	::= $\langle \text{sign} \rangle^* \langle \text{digit} \rangle^+$
11 $\langle \text{number} \rangle$	::= $\langle \text{sign} \rangle^? \langle \text{significand} \rangle \langle \text{exponent} \rangle^?$
12 $\langle \text{significand} \rangle$	::= ('.'? $\langle \text{digit} \rangle^+$) ($\langle \text{digit} \rangle^+$ '.' $\langle \text{digit} \rangle^*$)
13 $\langle \text{exponent} \rangle$	::= ('e' 'E') $\langle \text{sign} \rangle^? \langle \text{digit} \rangle^+$
14 $\langle \text{other} \rangle$	::= $\langle \text{number} \rangle$ '+' '-' '*' '/'

$\backslash \text{c_bnvs_i_regex}$ Signed integer.

(End of definition for $\backslash \text{c_bnvs_i_regex}$.)

```

17 \regex_const:Nn \c__bnvs_i_regex {
18   [-+]* \d+
19 }

```

$\backslash \text{c_bnvs_dot_i_regex}$ Signed integer with a dot . just before.

(End of definition for $\backslash \text{c_bnvs_dot_i_regex}$.)

```

20 \regex_const:Nn \c__bnvs_dot_i_regex {
21   \. \ur{c__bnvs_i_regex}
22 }

```

`\c__bnvs_A_dot_index_Z_regex` Full signed integer with a dot . jus before.

(End of definition for \c__bnvs_A_dot_index_Z_regex.)

```

23 \regex_const:Nn \c__bnvs_A_dot_index_Z_regex {
24   \A \ur { c__bnvs_dot_i_regex } \Z
25 }

```

`\c__bnvs_A_i_Z_regex` Signed integer with no capture groups.

(End of definition for \c__bnvs_A_i_Z_regex.)

```

26 \regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_i_regex} \Z }

```

`\c__bnvs_dot_R_regex` A possibly empty list of $\langle \text{short name} \rangle$ items representing a path. Integers are allowed as well.

```

27 \regex_const:Nn \c__bnvs_R_regex {
28   \ur{c__bnvs_i_regex} | [[:alnum:]]_+
29 }

```

(End of definition for \c__bnvs_dot_R_regex.)

`\c__bnvs_dot_R_regex` A possibly empty list of $\langle \text{short name} \rangle$ items representing a path. Integers are allowed as well.

```

30 \regex_const:Nn \c__bnvs_dot_R_regex {
31   \. \ur{c__bnvs_R_regex}
32 }

```

(End of definition for \c__bnvs_dot_R_regex.)

Then we define identifiers. $\langle S \rangle$ stands for a short identifier, $\langle P \rangle$ stands for a path component and $\langle R \rangle$ stands for a relative component.

```

15  $\langle S \rangle$       ::=  $\langle \text{alpha} \rangle \langle \text{alnum} \rangle^*$ 
16  $\langle P \rangle$       ::=  $\langle \text{digit} \rangle^* \langle S \rangle$ 
17  $\langle R \rangle$       ::= (  $\langle \text{'.'} \rangle \langle P \rangle$  ) | (  $\langle \text{'.'} \rangle \langle \text{index} \rangle$  ) |  $\langle N \rangle$ 
18  $\langle N \rangle$       ::=  $\langle \text{'['} \rangle \langle \text{Rhs} \rangle \langle \text{']'} \rangle$ 
19  $\langle \text{IKSPR} \rangle$  ::= ( (  $\langle S \rangle ? \langle \text{'!' } \rangle ? \langle S \rangle$  ) |  $\langle \text{'.'} \rangle \langle P \rangle$  )  $\langle R \rangle^*$ 
20  $\langle \text{Rhs} \rangle$    ::=  $\langle \text{Cr1} \rangle$  |  $\langle \text{Sqr} \rangle$  |  $\langle \text{VAZL} \rangle$  |  $\langle \text{VAZLRnd} \rangle$ 

```

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```

33 \regex_const:Nn \c__bnvs_S_regex {
34   [[:alpha:]]_+ [[:alnum:]]_*
35 }

```

(End of definition for \c__bnvs_S_regex.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

36 \regex_const:Nn \c__bnvs_P_regex {
37   \d*[[[:alpha:]]_][[:alnum:]]_*
38 }

```

(End of definition for `\c__bnvs_P_regex`.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```

39 \regex_const:Nn \c__bnvs_dot_P_regex {
40   \. \ur{c__bnvs_P_regex}
41 }

```

(End of definition for `\c__bnvs_dot_P_regex`.)

The lists between curly delimiters are used for $\langle \text{key} \rangle = \langle \text{value} \rangle$ definitions, with optional values. Each $\langle \text{CrLI} \text{tm} \rangle$ end with either a comma, an unbalanced closing delimiter or `\q_stop`. Here $\langle \text{Rhs} \rangle$ ends exactly the same.

```

21 \langle \text{CrI} \rangle      ::= '{' \langle \text{CrLI} \text{st} \rangle '}'
22 \langle \text{CrLI} \text{st} \rangle ::= \langle \text{space} \rangle \langle \text{CrLI} \text{tm} \rangle ( \langle \text{comma} \rangle \langle \text{CrLI} \text{tm} \rangle )* \langle \text{space} \rangle
23 \langle \text{CrLI} \text{tm} \rangle   ::= \langle \text{CrLEq1} \rangle | \langle \text{IKSPR} \rangle
24 \langle \text{CrLEq1} \rangle   ::= ( \langle \text{IKSPR} \rangle \langle \text{equal} \rangle )+ \langle \text{Rhs} \rangle

```

The lists between square delimiters are used for $\langle \text{key} \rangle = \langle \text{value} \rangle$ definitions, with optional keys.

```

25 \langle \text{Sqr} \rangle      ::= '[' \langle \text{SqrLI} \text{st} \rangle ']'
26 \langle \text{SqrLI} \text{st} \rangle ::= \langle \text{space} \rangle \langle \text{SqrITm} \rangle ( \langle \text{comma} \rangle \langle \text{SqrITm} \rangle )* \langle \text{space} \rangle
27 \langle \text{SqrITm} \rangle   ::= \langle \text{SqrEq1} \rangle | \langle \text{Rhs} \rangle
28 \langle \text{SqrEq1} \rangle   ::= ( ( \langle \text{IKSPR} \rangle | \langle \text{index} \rangle ) \langle \text{equal} \rangle )+ \langle \text{Rhs} \rangle

```

For individual ranges or values:

```

29 \langle \text{VAZL} \rangle      ::= \langle \text{Blnc} \rangle | ( \langle \text{Blnc} \rangle ':' \langle \text{Blnc} \rangle? ( ':' \langle \text{Blnc} \rangle? )? )
30                               | ( \langle \text{Blnc} \rangle "::" \langle \text{Blnc} \rangle? ( ':' \langle \text{Blnc} \rangle? )? )
31 \langle \text{Blnc} \rangle      ::= ( \langle \text{space} \rangle | \langle \text{IKSPR} \rangle | \langle \text{Rnd} \rangle | \langle \text{Sqr} \rangle | \langle \text{CrI} \rangle | \langle \text{Rlv} \rangle | \langle \text{Raw} \rangle |
32   \hookrightarrow \langle \text{other} \rangle ) *
33 \langle \text{VAZLRnd} \rangle ::= '(' \langle \text{VAZLLst} \rangle ')'
34 \langle \text{VAZLLst} \rangle ::= \langle \text{space} \rangle \langle \text{VAZL} \rangle ( \langle \text{comma} \rangle \langle \text{VAZL} \rangle )* \langle \text{space} \rangle
35 \langle \text{Rlv} \rangle        ::= '?' \langle \text{VAZLRnd} \rangle

```

For function calls,

```

35 <Raw>      ::= ' ' ' (' <RawLst> ')'
36 <RawLst>   ::= ( <space> | <Rlv> | <RawRnd> | <RawSqr> | <RawCrl> | <RawOtr>
    ↪ ) *
37 <RawRnd>   ::= '(' <RawLst> ')'
38 <RawSqr>   ::= '[' <RawLst> ']'
39 <RawCrl>   ::= '{' <RawLst> '}'
40 <RawOtr>   ::= <comma> | <alnum> | <other>

```

1.3.2 Storage

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual “\”, “{” and “}” before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use $\TeX$ groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

There is also a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

```

\BNVS_xprt_reset: \BNVS_xprt_reset:
\BNVS_xprt:n      \BNVS_xprt:n {<scanned>}
\BNVS_scan_xprt:n \BNVS_scan_xprt:n {<scanned>}

```

`\BNVS_xprt:n` is a utility function to store its argument in the `scanned` variable. Automatically set by `\BNVS_begin_scan:...` functions. Used by `\BNVS_end_scan:..` The default implementation just raises. `\BNVS_xprt_crl:n` feeds the scan storage with `<scanned>`, enclosed between one pair of curly braces. `\BNVS_scan_xprt:n` is called before exporting. The argument is enclosed inside `\exp_not:n{...}`. The default implementation just expands to its argument as is.

```

42 \cs_new:Npn \BNVS_xprt_reset: {
43   \__bnvs_tl_clear:c { scanned }
44 }

45 \cs_new:Npn \BNVS_xprt:n #1 {
46   \BNVS_xprt_stored:
47   \__bnvs_tl_put_right:cn { scanned } { #1 }
48 }

49 \cs_new_eq:NN \BNVS_scan_xprt:n \use:n

```

```

\BNVS_xprt_crl:n \BNVS_xprt_crl:n {<scanned>}
\BNVS_xprt_use:n \BNVS_xprt_use:n {<code:n>}

```

`\BNVS_xprt_reset:` resets the scan storage, called when starting a new $\TeX$ group dedicated to scanning. Implementation detail: the second is based on the first. `\BNVS_xprt_use:n` puts back to the input stream the scanned material enclosed in one pair of braces after the instructions `<code:n>`.

```

50 \cs_new:Npn \BNVS_xprt_use:n #1 {
51   \BNVS_xprt_stored:
52   \BNVS_tl_use:nv { #1 } { scanned }
53 }

54 \cs_new:Npn \BNVS_xprt_crl:n #1 {
55   \BNVS_xprt:n { { #1 } }
56 }

```

\BNVS_scan_wrap:n	\BNVS_scan_wrap:n {<function:w>}
\BNVS_scan_wrap_xprt:n	\BNVS_scan_wrap_xprt:n {<body:n>}

\BNVS_scan_wrap:n sets the function \BNVS_scan_wrapper:w to expand to \exp_not:n { <wrapped:w> }
This is used in to define \BNVS_scan_xprt:n, when the arguments must be scanned prior
to knowing which function <wrapped:w> fits.

```

57 \cs_new:Npn \BNVS_scan_wrapper:w {}

58 \cs_new:Npn \BNVS_scan_wrap:n #1 {
59   \cs_set:Npn \BNVS_scan_wrapper:w { \exp_not:n { #1 } }
60 }

61 \cs_new:Npn \BNVS_scan_wrap_xprt:n #1 {
62   \cs_set:Npn \BNVS_scan_xprt:n ##1 {
63     \tl_if_empty:nF { ##1 } {
64       \BNVS_scan_wrapper:w
65       \exp_not:n { #1 }
66     }
67   }
68 }

```

\BNVS_xprt_store:n	\BNVS_xprt_store:n {<something>}
\BNVS_xprt_stored:	\BNVS_xprt_stored:

\BNVS_xprt_store:n synchronously cumulates its argument in the stored variable.
The function \BNVS_xprt_stored: is executed both at the beginning of the function
\BNVS_xprt:n and when a scan group ends. It exports the contents of the stored
<material>, when non empty, to the scanned variable as \:n{<material>} instruction
and cleans the stored variable.

```

69 \cs_new:Npn \BNVS_xprt_stored: {
70   \__bnvs_tl_if_empty:cF { stored } {
71     \BNVS_tl_use:nv {
72       \__bnvs_tl_clear:c { stored }
73       \BNVS_xprt:n { \:n }
74       \BNVS_xprt_crl:n
75     } { stored }
76   }
77 }

78 \cs_new:Npn \BNVS_xprt_store:n #1 {
79   \tl_if_empty:nF { #1 } {
80     \__bnvs_tl_put_right:cn { stored } { #1 }
81   }
82 }

```

```
\BNVS_next_set:n \BNVS_next_set:n {<next:>}
\BNVS_next_use:n \BNVS_next_use:n {<code:>}
```

`\BNVS_next_set:n` stores `<next:>` to be executed later by `\BNVS_next_use:n`, just after `<code:>`. Typically, `\BNVS_next_set:n` is used in a scan group closed by `<code:>`. `\BNVS_end_scan_next:` ends the scan and executes `<next:>` code.

```
83 \BNVS_tl_new:c { next }
84 \cs_new:Npn \BNVS_next_set:n #1 {
85   \__bnvs_tl_set:cn { next } {
86     \exp_not:n {
87       #1
88     }
89   }
90 }

91 \cs_new:Npn \BNVS_next_use:n #1 {
92   \BNVS_tl_last_unbraced:nv { #1 } { next }
93 }
```

```
\BNVS_begin_scan:nnnnnnn \BNVS_begin_scan:nnnnnnn {<xprt:n>} {<next:>}
                           {<closed:>} {<comma:>} {<one colon:>} {<colons:>} {<stopped:>}
```

Start a scanning group with all branch functions definitions.

❗ One shot utility only used by `\BNVS_begin_scan:nnnnnnn` and friends.

```
94 \cs_new:Npn \BNVS_scan_begun:nnnnnnn #1 #2 #3 #4 #5 #6 #7 {
95   \BNVS_xprt_reset:
96   \BNVS_scan_wrap_xprt:n { #1 }
97   \BNVS_next_set:n {
98     #2
99   }
100   \cs_set:Npn \BNVS_scanned_close: {
101     #3
102   }
103   \cs_set:Npn \BNVS_scanned_comma: {
104     #4
105   }
106   \cs_set:Npn \BNVS_scanned_one_colon: {
107     #5
108   }
109   \cs_set:Npn \BNVS_scanned_colons: {
110     #6
111   }
112   \cs_set:Npn \BNVS_scanned_stop: {
113     #7
114   }
115 }

116 \cs_new:Npn \BNVS_begin_scan:nnnnnnn {
117   \BNVS_begin:
118   \BNVS_scan_begun:nnnnnnn
119 }
```

<code>\BNVS_end_scan:</code>	<code>\BNVS_end_scan:</code>
<code>\BNVS_end_scan:n</code>	<code>\BNVS_end_scan:n {\prepare:e}</code>
<code>\BNVS_end_scan_next:</code>	

The function `\BNVS_end_scan:` must be called to close the $\TeX$ scan group, it calls `\BNVS_xprt:n` on the scanned material.

```

120 \cs_new:Npn \BNVS_end_scan: {
121   \BNVS_xprt_use:n {
122     \BNVS_end_scan:n
123   }
124 }

125 \cs_new:Npn \BNVS_end_scan:n #1 {
126   \exp_args:NNx \BNVS_end: \BNVS_xprt:n {
127     \BNVS_scan_xprt:n { #1 }
128   }
129 }

130 \cs_new:Npn \BNVS_end_scan_next: {
131   \BNVS_next_use:n { \BNVS_end_scan: }
132 }
```

<code>\BNVS_begin_scan:nn</code>	<code>\BNVS_begin_scan:nn {\xprt:n}</code>
	<code>{\next:n}</code>

Call `\BNVS_begin_scan:nnnnnnn` with default arguments that end the scan group and call the eponym branch function.

```

133 \cs_new:Npn \BNVS_begin_scan:nn #1 #2 {
134   \BNVS_begin_scan:nnnnnnn { #1 } { #2 }
135   { \BNVS_end_scan: \BNVS_scanned_close: }
136   { \BNVS_end_scan: \BNVS_scanned_comma: }
137   { \BNVS_end_scan: \BNVS_scanned_one_colon: }
138   { \BNVS_end_scan: \BNVS_scanned_colons: }
139   { \BNVS_end_scan: \BNVS_scanned_stop: }
140 }
```

In the sequel, $\langle input:w \rangle$ is expected to read the $\langle input\ stream \rangle$ until a `\q_stop` token that is gobbled.

<code>_bnvs_scan_q_stop:F</code>	<code>_bnvs_scan_q_stop:F {\input:w} \langle input stream \rangle</code>
<code>\BNVS_scanned_stop:</code>	

On scanning a `\q_stop` from the head of the $\langle input\ stream \rangle$, calls the `\BNVS_scanned_stop:` branch function, otherwise execute $\langle input:w \rangle$.

```

141 \BNVS_new:cpn { scan_q_stop:F } {
142   \peek_meaning_remove:NTF \q_stop {
143     \BNVS_scanned_stop:
144   }
```

⊘ There must be absolutely no code here. No hooking allowed either.

```

145 }
```

⚡ Do nothing operation at the top level.

```

146 \cs_set:Npn \BNVS_scanned_stop: {}
```

<code>_bnvs_scan_comma:Fw</code> <code>\BNVS_scanned_comma:</code>	<code>_bnvs_scan_comma:Fw</code> $\langle input:w \rangle$ $\langle input\ stream \rangle$ On scanning a comma from the head of the $\langle input\ stream \rangle$, calls <code>\BNVS_scanned_comma:</code> branch function, otherwise execute $\langle input:w \rangle$. Beware, it gobbles spaces.
--	---

```

147 \BNVS_new:cpn { scan_comma:Fw } #1 {
148   \peek_remove_spaces:n {
149     \peek_charcode_remove:NTF , {
150       \peek_remove_spaces:n { \BNVS_scanned_comma: }
151     } {
152       #1
153     }
154   }
155 }

```

¶ Do nothing operation at the top level.

```

156 \cs_set:Npn \BNVS_scanned_comma: {
157 }

```

<code>_bnvs_scan_colon:Fw</code> <code>\BNVS_scanned_one_colon:</code> <code>\BNVS_scanned_colons:</code>	<code>_bnvs_scan_colon:Fw</code> $\{\langle input:w \rangle\}$ $\langle input\ stream \rangle$ On scanning a standalone colon from the $\langle input\ stream \rangle$, calls the branch function <code>\BNVS_scanned_one_colon:w</code> . On scanning multiple colons from the $\langle input\ stream \rangle$, calls the <code>\BNVS_scanned_colons:w</code> branch function. In other cases, execute the $\langle input:w \rangle$.
--	---

```

158 \BNVS_new:cpn { scan_colon:Fw } #1 {
159   \peek_remove_spaces:n {
160     \peek_charcode_remove:NTF : {
161       \peek_charcode_remove:NTF : {
162         \peek_remove_spaces:n { \BNVS_scanned_colons:w }
163       } {
164         \peek_remove_spaces:n { \BNVS_scanned_one_colon:w }
165       }
166     } {
167       #1
168     }
169   }
170 }

```

<code>_bnvs_scan_close:Fw</code> <code>\BNVS_scanned_close:N</code> <code>\BNVS_scanned_close:</code>	<code>_bnvs_scan_close:Fw</code> $\langle no:w \rangle$ $\langle input\ stream \rangle$ <code>\BNVS_scanned_close:N</code> $\langle clositer \rangle$ $\langle input\ stream \rangle$ When a closing delimiter is scanned, call <code>\BNVS_scanned_close:N</code> with that delimiter as argument. Otherwise execute $\langle no:w \rangle$ code.
--	---

`\BNVS_scanned_close:N` is a branch function called when the closing delimiter $\langle clositer \rangle$ has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:` used many times. When re-defined (for example by `\_bnvs_scan_delimited:NNnFw`), the `\BNVS_scanned_close:` function does not forward to the eponym function, instead it must know where to go next.

```

171 \group_begin:
172 \char_set_catcode_group_begin:N <
173 \char_set_catcode_group_end:N >
174 \char_set_catcode_other:N \{
175 \char_set_catcode_other:N \}
176 \BNVS_new:cpn < scan_close:Fw > #1 <
177   \peek_charcode_remove:NTF%(
178   ) <
179   \BNVS_scanned_close:N%(
180   )
181   > <
182   \peek_charcode_remove:NTF%[
183   ] <
184   \BNVS_scanned_close:N%[
185   ]
186   > <
187   \peek_charcode_remove:NTF %{{
188   } <
189   \BNVS_scanned_close:N %{{
190   }
191   > <
192   #1
193   >
194   >
195   >
196   >
197 \group_end:

198 \cs_set:Npn \BNVS_scanned_close:N #1 {
199   \BNVS_error:n { Unexpected~delimiter~`#1' }
200   \BNVS_scanned_close:
201 }

❗ Do nothing operation at the top level.

202 \cs_set:Npn \BNVS_scanned_close: {
203 }

```

\BNVS_begin_scan:nn	\BNVS_begin_scan:nn {⟨closed:⟩} {⟨stopped:⟩}
\BNVS_begin_scan:nnn	\BNVS_begin_scan:nnn {⟨closed:⟩} {⟨comma:⟩} {⟨stopped:⟩}
\BNVS_begin_scan:nnnn	\BNVS_begin_scan:nnnn {⟨closed:⟩} {⟨comma:⟩} {⟨one colon:⟩} {⟨colons:⟩}
	{⟨stopped:⟩}

Start a scanning group with various branch functions definitions.

1.3.3 Comma separated lists

\BNVS_scan_csl_item:w	\BNVS_scan_csl_item:w ⟨input stream⟩
-----------------------	--------------------------------------

Scan a single item in a comma separated list. The item ends at the first top level comma, the first unbalanced delimiter if one is found, or when a `\q_stop` is found. The `stored` variable is fed with material that cannot be processed by `⟨scan:Fw⟩` function.

```

204 \cs_set:Npn \BNVS_scan_csl_item:w {
205   \BNVS_scan_special:TFw {
206     \BNVS_scan_csl_item:w
207   } {
208     \__bnvs_scan_comma:Fw {
209       \__bnvs_scan_close:Fw {
210         \__bnvs_scan_q_stop:F {
211           \__bnvs_scan_csl_other:Nw
212         }
213       }
214     }
215   }
216 }

217 \cs_new:Npn \BNVS_scan_special:TFw #1 #2 {
218   \BNVS_error:n { \BNVS_scan_special:TFw~undefined~at~top~level. }
219 }

220 \BNVS_new:cpn { scan_csl_other:Nw } #1 {
221   \BNVS_xprt_store:n { #1 }
222   \BNVS_scan_csl_item:w
223 }

```

`\BNVS_scan_csl:nnw` `\BNVS_scan_csl:nnw {<closed:>} {<stopped:>} <input stream>`

This function scans the comma separated list of items, regardless the enclosing delimiters. It begins a scanning $\text{\TeX}$ group that is ended by a call to `\BNVS_scanned_close:`, executing `<closed:>`, or `\BNVS_scanned_stop:`, executing `<stop:>`. The scanned material is then exported between braces. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input `a` it is `\:nn{a}{}`, for input `a,b` it is `\:nn{a}{\:nn{b}{}}`, and so on.

```

224 \cs_new:Npn \BNVS_scan_csl:nnw #1 #2 {
225   \BNVS_begin_scan:nnnnnnn
226   { ##1 }
227   { }

```

❧ Close branch.

```

228   { \BNVS_end_scan: #1 }
229   {
230     \BNVS_xprt_use:n { \tl_if_empty:nTF } {

```

❧ Comma branch, the first item is empty, ignore it.

```

231     \BNVS_scan_csl_item:w
232   } {

```

❧ Comma branch, the first item is not empty, there will be another item, possibly empty.

```

233     \BNVS_scan_wrap_xprt:n { { ####1 } }
234     \BNVS_end_scan:
235     \BNVS_begin_scan:nnnnnnn

```

```

236         { { ####1 } }
237         { }
238         { \BNVS_end_scan: \BNVS_scanned_close: }
239         { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
240         { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
241         { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
242         { \BNVS_end_scan: \BNVS_scanned_stop: }
243     \BNVS_scan_csl:nnw {
❧ Close branch.
244         \BNVS_xprt_use:n { \tl_if_empty:nTF } {
245             \BNVS_end_scan:
246             \BNVS_xprt:n { { } }
247         } {
248             \BNVS_end_scan:
249         }
250         \BNVS_scanned_close:
251     } {
❧ Stop branch.
252         \BNVS_xprt_use:n { \tl_if_empty:nTF } {
253             \BNVS_end_scan:
254             \BNVS_xprt:n { { } }
255         } {
256             \BNVS_end_scan:
257         }
258         \BNVS_scanned_stop:
259     }
260 }
261 }
262 { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
263 { \BNVS_error:n { Unreachable } \BNVS_scanned_close: }
264 { \BNVS_end_scan: #2 }

❧ By default, the list contains only one item.
265     \BNVS_scan_wrap:n { \:nn }
266     \BNVS_scan_wrap_xprt:n { { ##1 } { } }
267     \BNVS_scan_csl_item:w
268 }

```

1.3.4 Delimiters

We assume that delimiters are paired and balanced. Nevertheless, some errors are caught and properly managed.

```

\BNVS_begin_scan_close:nnNn \BNVS_begin_scan_close:nnNn {<xprt:n>} {<next:n>} <close> {<mtrl:w>}
\BNVS_begin_scan_close:NnNn \BNVS_begin_scan_close:NnNn <xprt:n> {<next:n>} <close> {<mtrl:w>}

```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter. Implementation detail: we begin a scanning group that will be closed on scanning the `<close>` delimiter, or a different closing delimiter, allways at the same level. The `<mtrl:w>` function scans the `<input stream>`. This is the normal instruction flow.

```

269 \cs_new:Npn \BNVS_begin_scan_close:nnNn {
270   \BNVS_begin:
271   \BNVS_scan_begun_close:nnNn
272 }

273 \cs_new:Npn \BNVS_begin_scan_close:NnNn #1 {
274   \BNVS_begin_scan_close:nnNn { #1 { ##1 } }
275 }

❗ Helper only used by \BNVS_begin_scan_close:<[nN]>Nn.

276 \cs_new:Npn \BNVS_scan_begun_close:nnNn #1 #2 #3 {
277   \cs_set:Npn \BNVS_scanned_close:N ##1 {
278     \tl_if_eq:nnF { #3 } { ##1 } {
❗ If the closing delimiter scanned is not the expected one, raise.

279       \BNVS_error:n { Unexpected~`##1'~instead-of~`#3'. }
280     }

❗ Anyways, branch here.

281     \BNVS_end_scan_next:
282   }
283   \BNVS_scan_begun:nnnnnn { #1 } { #2 }

❗ Close branch. End the group and execute <next:> instructions. Intermediate groups
should be closed appropriately with the right definition of branch functions.

284   { \BNVS_end_scan_next: }

❗ Any other branch function raises and forwards to \BNVS_scanned_close:,

285   { \BNVS_error:n { Missing~`#3'. } \BNVS_scanned_close: }
286   { \BNVS_error:n { Missing~`#3'. } \BNVS_scanned_close: }
287   { \BNVS_error:n { Missing~`#3'. } \BNVS_scanned_close: }

❗ or \BNVS_scanned_stop: after the group ends when \q_stop is just scanned.

288   {
289     \BNVS_error:n { Missing~`#3'. }
290     \BNVS_end_scan:
291     \BNVS_scanned_stop:
292   }
293   \use:n
294 }

```